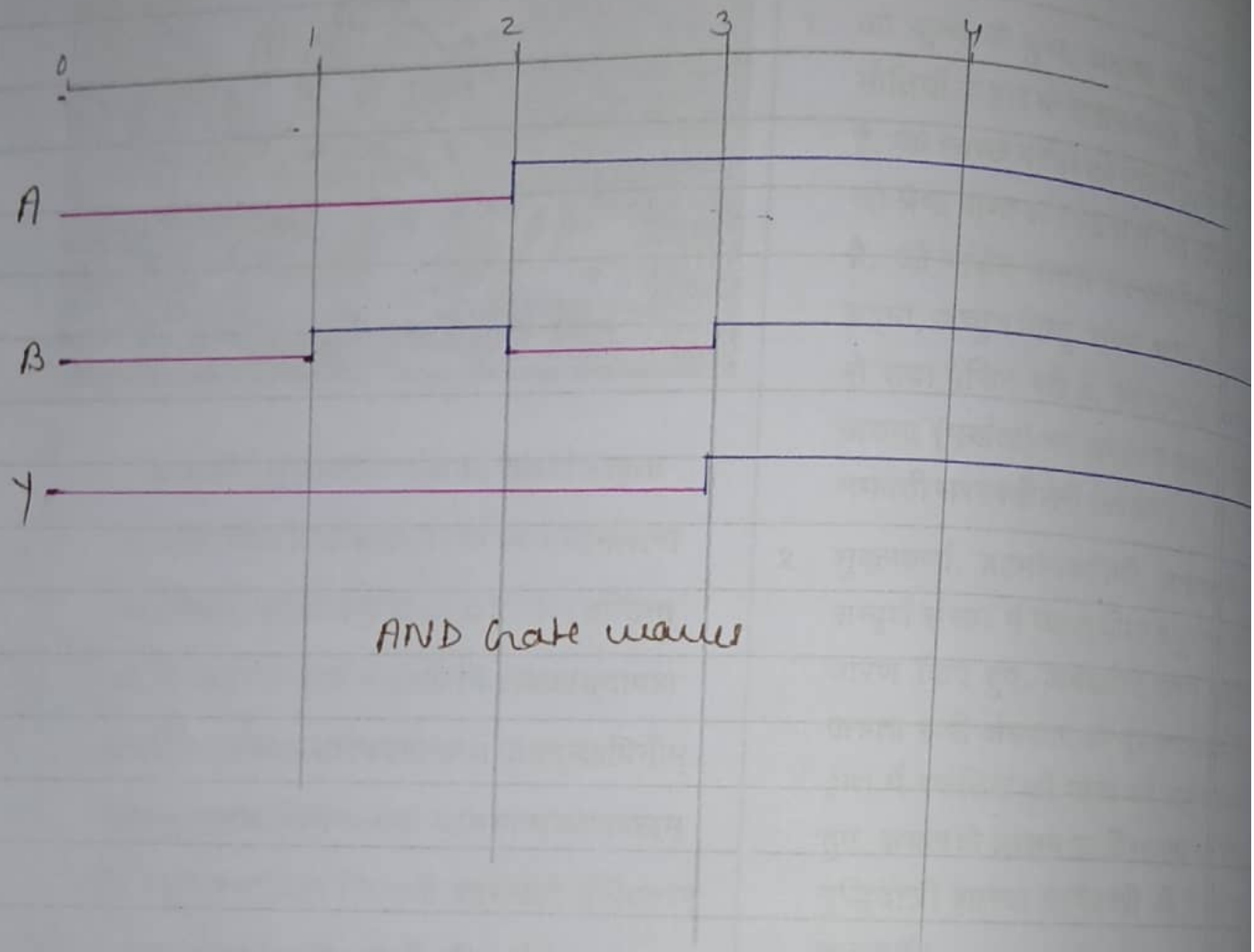


AND Gate waves

Scanned with CamScanner

POORNIMA

EXPERIment-1

* AND GATE

- Testbench.sv

```
module AND-GATE-tb;
    reg A;
    reg B;
    wire y;
    integer i;
    AND-Gate inst(.A(A), .B(B), .y(y));

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars;
        # 100 $finish;
    end
    initial
        begin
            A<=0;
            B<=0;
            $monitor("A=%ob B%:ob", A,B,y);
            for (i=0; i<4; i=i+1)
                begin
                    {A,B}=i;
            #10
                end
        end
endmodule
```


Scanned with CamScanner

POORNIMA

design.sv

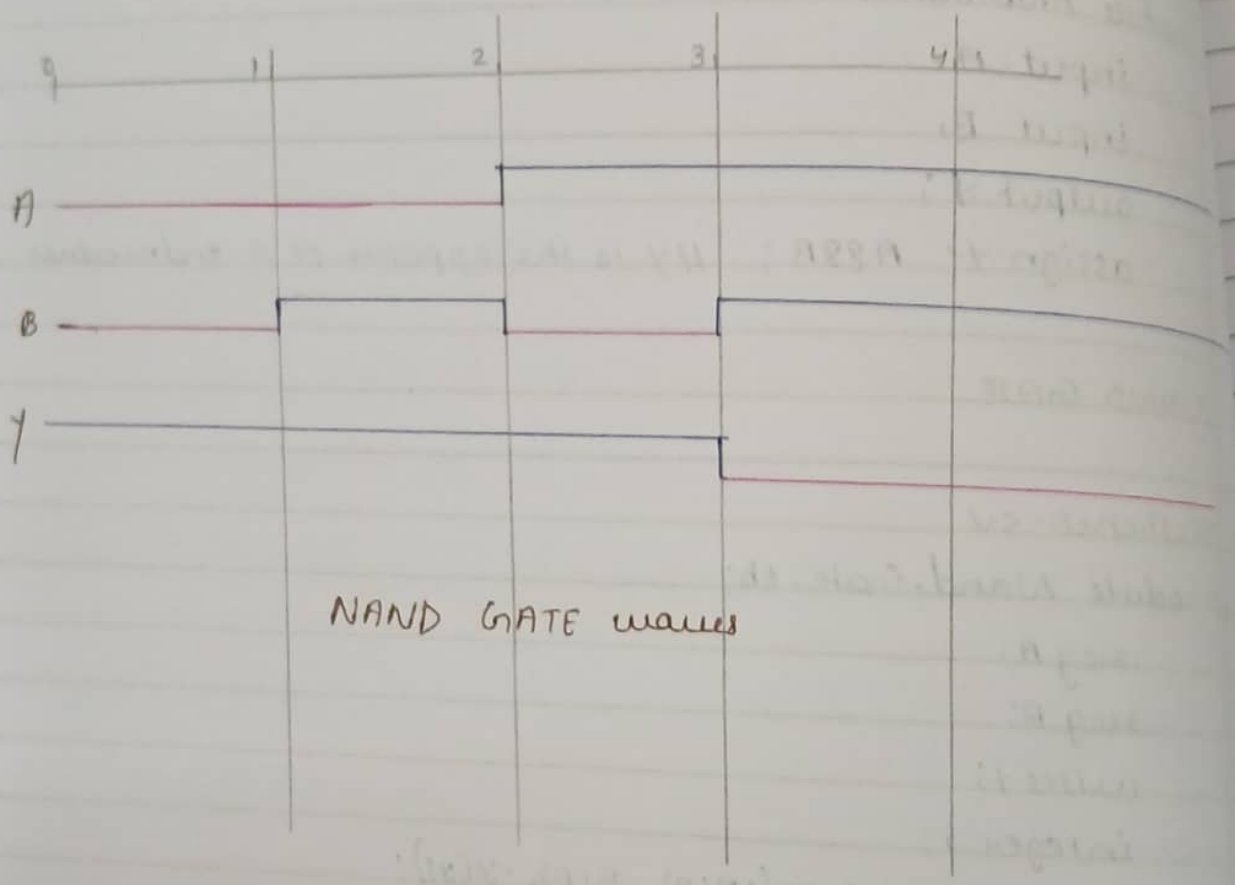
```
module AND_Gate(
    input A,
    input B,
    output y;
    assign y= A&&B; //y is the opposite of A endmodule
```

* NAND GATE

- Testbench.sv

```
module Nand_Gate_tb;
    reg A;
    reg B;
    wire y;
    integer i;
    Nand_Gate inst(.A(A), .B(B), .y(y));

  initial begin
    $dumpfile ("dump.vcd");
    $dumpvars;
    #100 $finish;
end
initial
  begin
    A<=0;
    B<=0;
```


Scanned with CamScanner



NAND GATE waves

Scanned with CamScanner

```
$monitor("A=%0b B=%0b y=%0b", A,B,y);
for (i=0; i<4; i=i+1)
    begin
    {A,B}=i;
#10
    end
  end
end module
```

design.sv

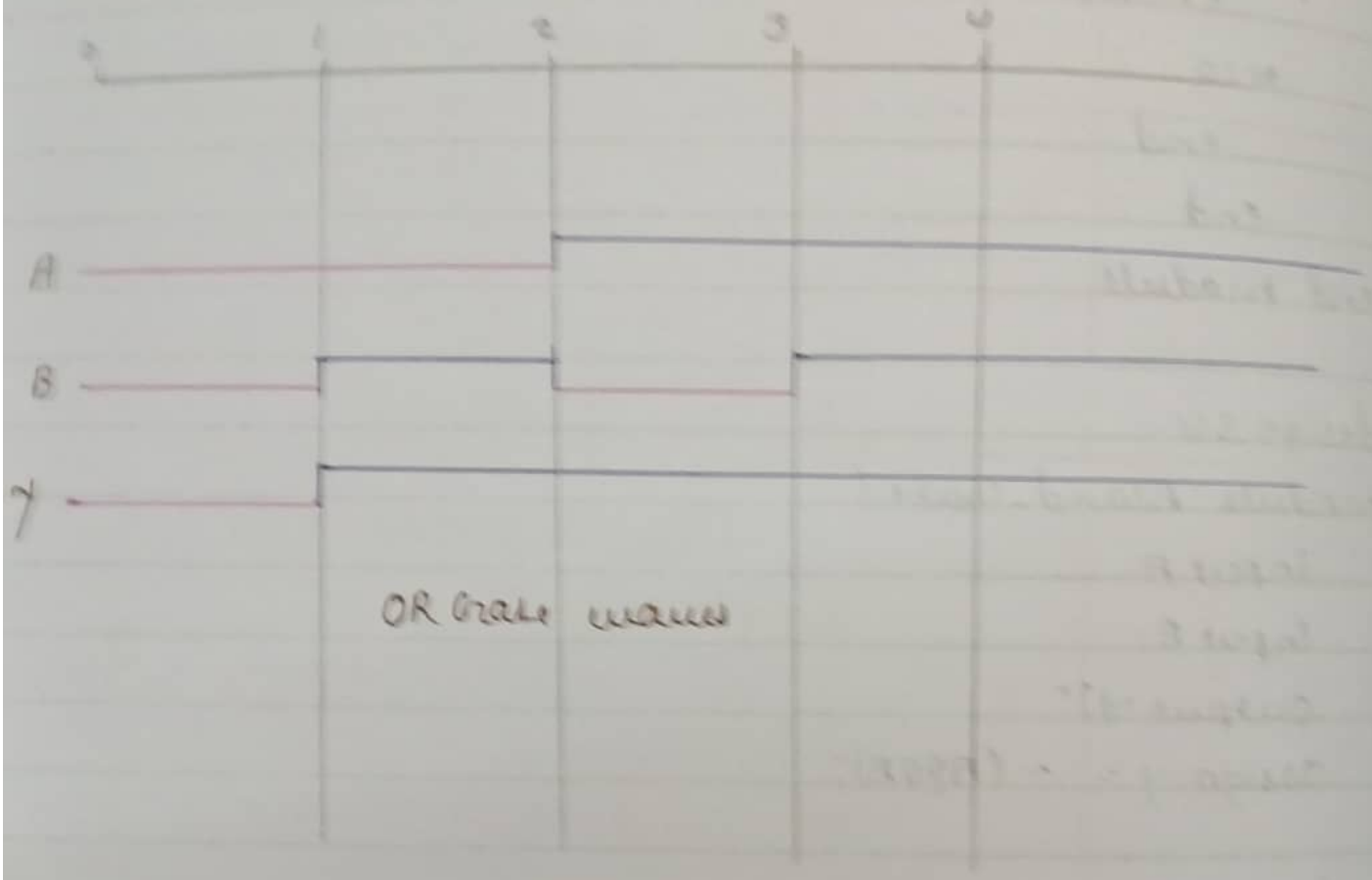
```
module Nand_Gate(
    input A,
    input B,
    output y);
  assign y = ~(A&&B);

end module
```

* OR Gate:-

Testbench.sv

```
module OR_Gate_tb;
    reg A;
    reg B;
    wire y;
    integer i;
    OR_gate inst(.A(A), .B(B), .y(y));
    initial begin
```


Scanned with CamScanner



Scanned with CamScanner

```
    $dumpfile("dump.vcd");
    $dumpvars;
    #100 $finish;
  end
  initial
    begin
      A<=0;
      B<=0
      $monitor("A=%ob B=%ob Y=%ob", A,B,Y);
      for (i=0; i<4; i=i+1)
        begin
          {A,B}=i;
      #10;
        end
    end
end module
```

Design.sv

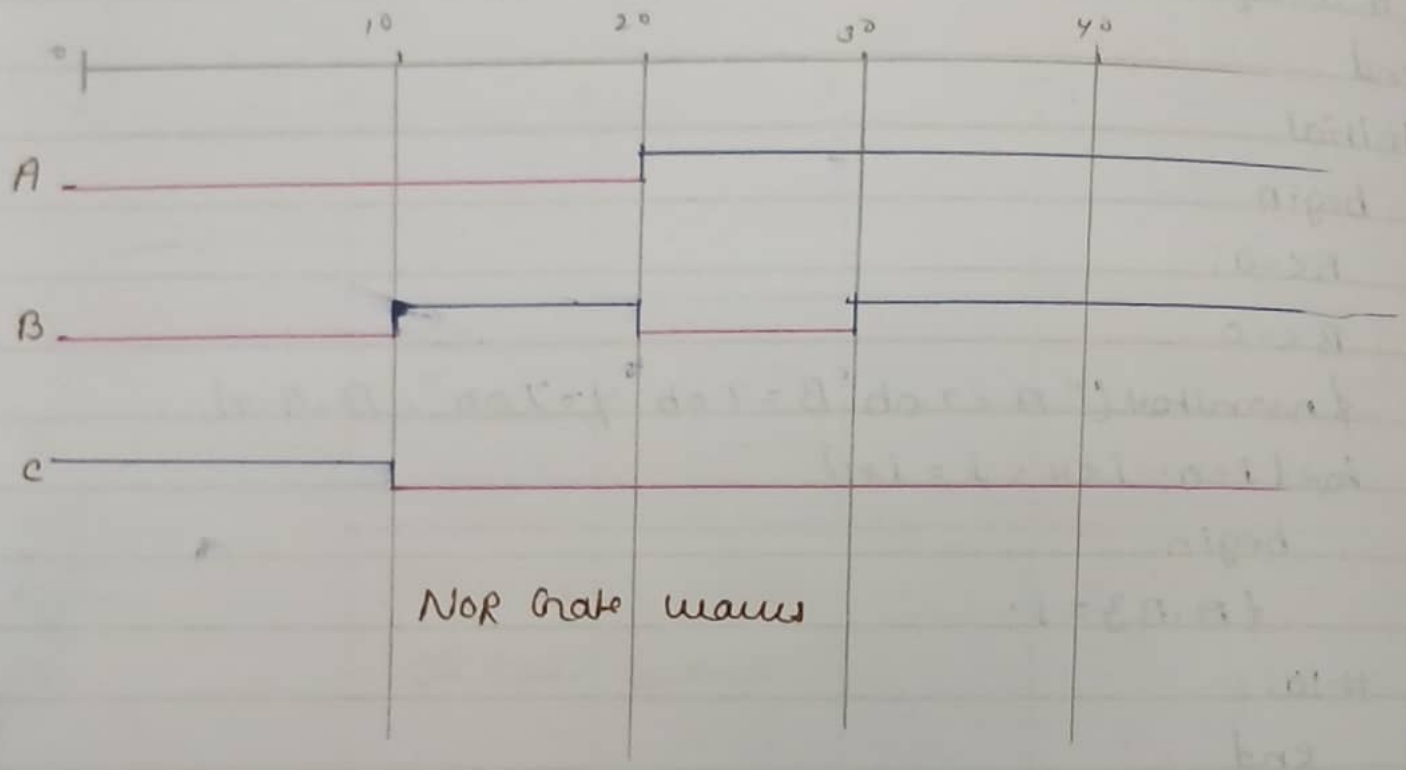
```
module OR-Gate(
    input A,
    input B,
    Output Y);
    assign Y= A||B;
endmodule
```

* NOR GATE

testbench.sv

```
module NOR_Gate_tb;
```


Scanned with CamScanner



Scanned with CamScanner

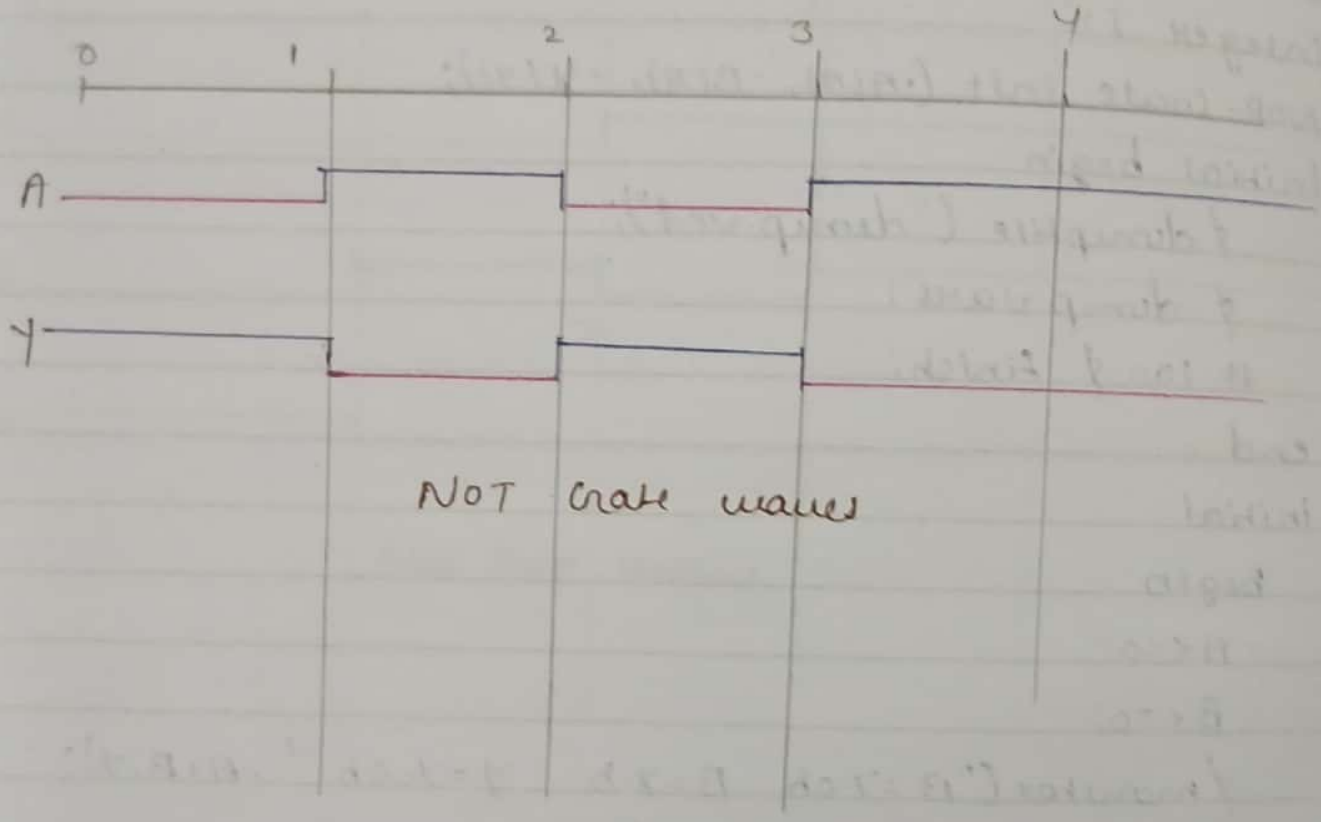
POORNIMA

```
reg A;
reg B;
wire y;
integer i;
NOR_Gate inst (.A(A), .B(B), .y(y));
initial begin
    $dumpfile ("dump.vcd");
    $dumpvars;
    #100 $finish;
end
initial
  begin
    A<=0;
    B<=0;
    $monitor ("A=%0b  B=%b  y=%0b", A,B,y);
    for (i=0; i<4; i=i+1)
      begin
        {A,B} = i;
    #10;
      end
    end
endmodule
```

Design.sv

```
module NOR_Gate (
    input A,
    input B,
    output y);
```


Scanned with CamScanner



NOT Gate waves

Scanned with CamScanner

```
assign y = ~(A||B);
end module
```

* NOT Gate

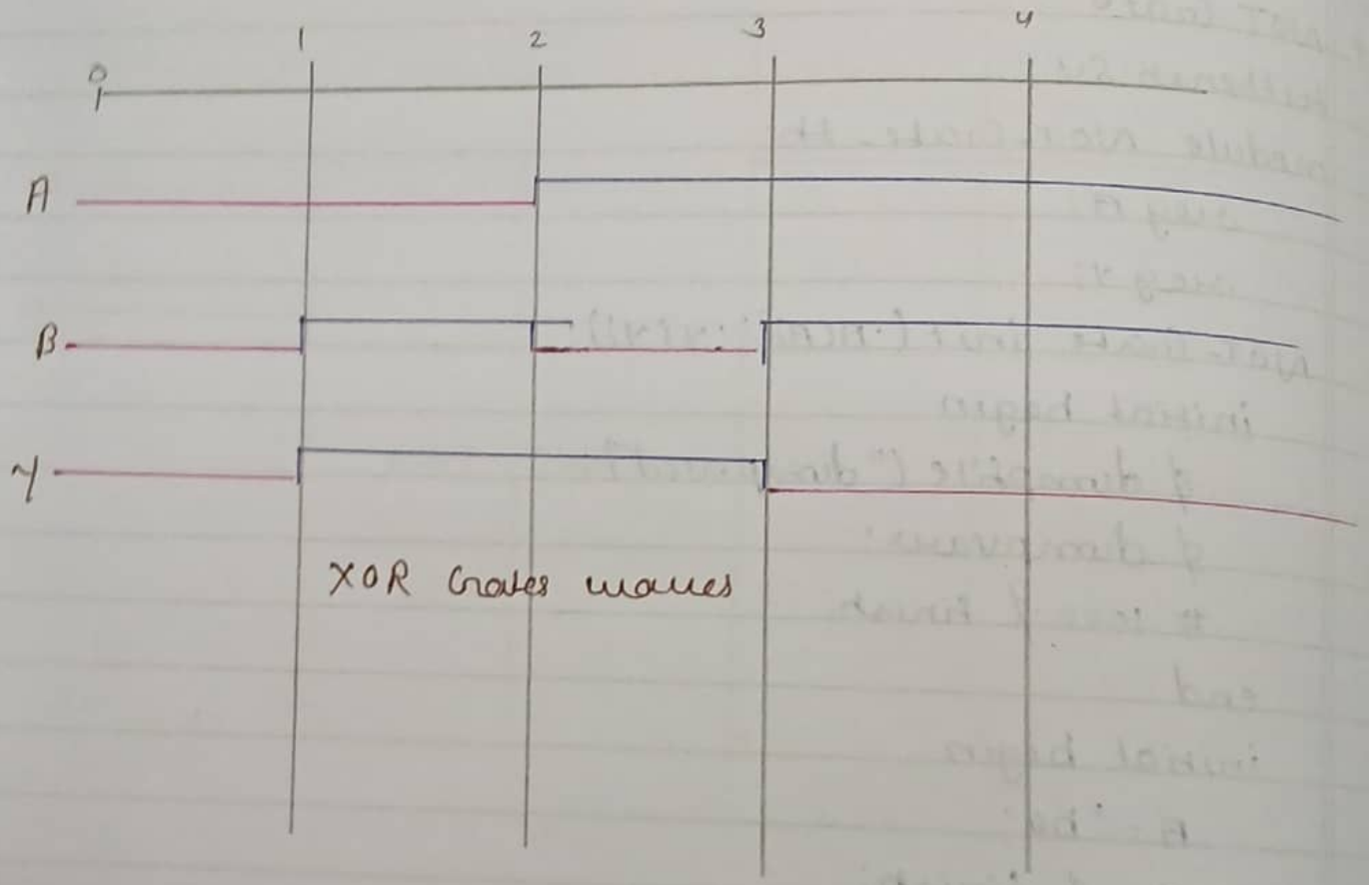
testbench.sv

```
module NOT-Gate_tb;
    reg A;
    reg y;
NOT-Gate inst(.A(A), .y(y));
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    #1000 $finish;
  end
  initial begin
    A = 'b0;
  #50 $finish;
 end
    always #5 A = ~A;
    always @(y)
      $display("time=%.4t : A=%.b\t output : y=%.b", $
                time, A, y);
endmodule
```

design.sv

```
module NOT-Gate(
    input A,
    output y);
```


Scanned with CamScanner



Scanned with CamScanner

POORNIMA

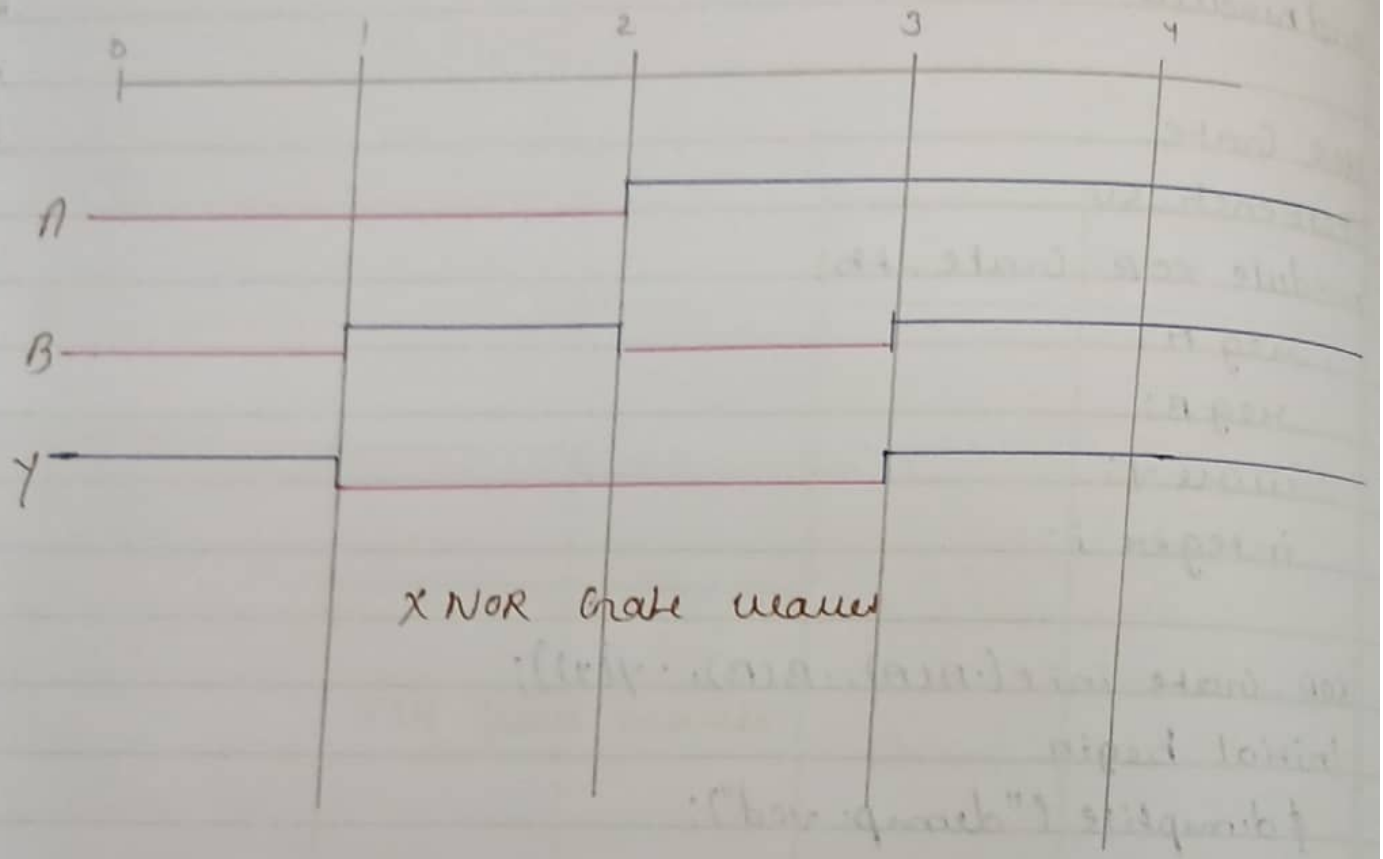
```
    assign y = ~A;
endmodule
```

* XOR Gate

Testbench.sv

```
module XOR_Gate_tb;
    reg A;
    reg B;
    wire y;
    integer i;

XOR_Gate inst(.A(A), .B(B), .y(y));
initial begin
  $dumpfile("dump.vcd");
  $dumpvars;
  #100 $finish;
end
initial
  begin
    A <= 0;
    B <= 0;
    $monitor("A=%0b B=%0b y=%0b", A, B, y);
    for (i=0; i<4; i=i+1)
      begin
        {A,B}=i;
    #10;
      end
  end
end module
```


Scanned with CamScanner



XNOR Gate waves

Scanned with CamScanner

POORNIMA

design.sv

```
module XOR_Gate(
    input A,
    input B,
    output y);
  assign y = ~A&&B || A&&~B;
end module
```

* XNOR Gate

testbench.sv

```
module XNOR_Gate_tb;
    regA;
    regB;
    wire y;
    integer i;
   XNOR_Gate inst(.A(A), .B(B), .y(y));
   initial begin
     $dumpfile("dump.vcd");
     $dumpvars;
    # 100 $ finish;
  end
   initial
    begin
      A<=0;
      B<=0;
     $ monitor("A=%ob B=%ob y=%ob", A,B,y);
     for(i=0; i<4; i=i+1)
     begin
```


Scanned with CamScanner

POORNIMA

```
{A,B} = i;
#10;
    end
  end
end module

design.sv
module XNOR-Gate(
    input A,
    input B,
    output y);
    assign y = ~(~A&&B || A&&~B);
end module.
```


Scanned with CamScanner

Half adder wave

Scanned with CamScanner

POORNIMA

EXPERIMENT-2

- OBJECTIVE:- HALF Adder

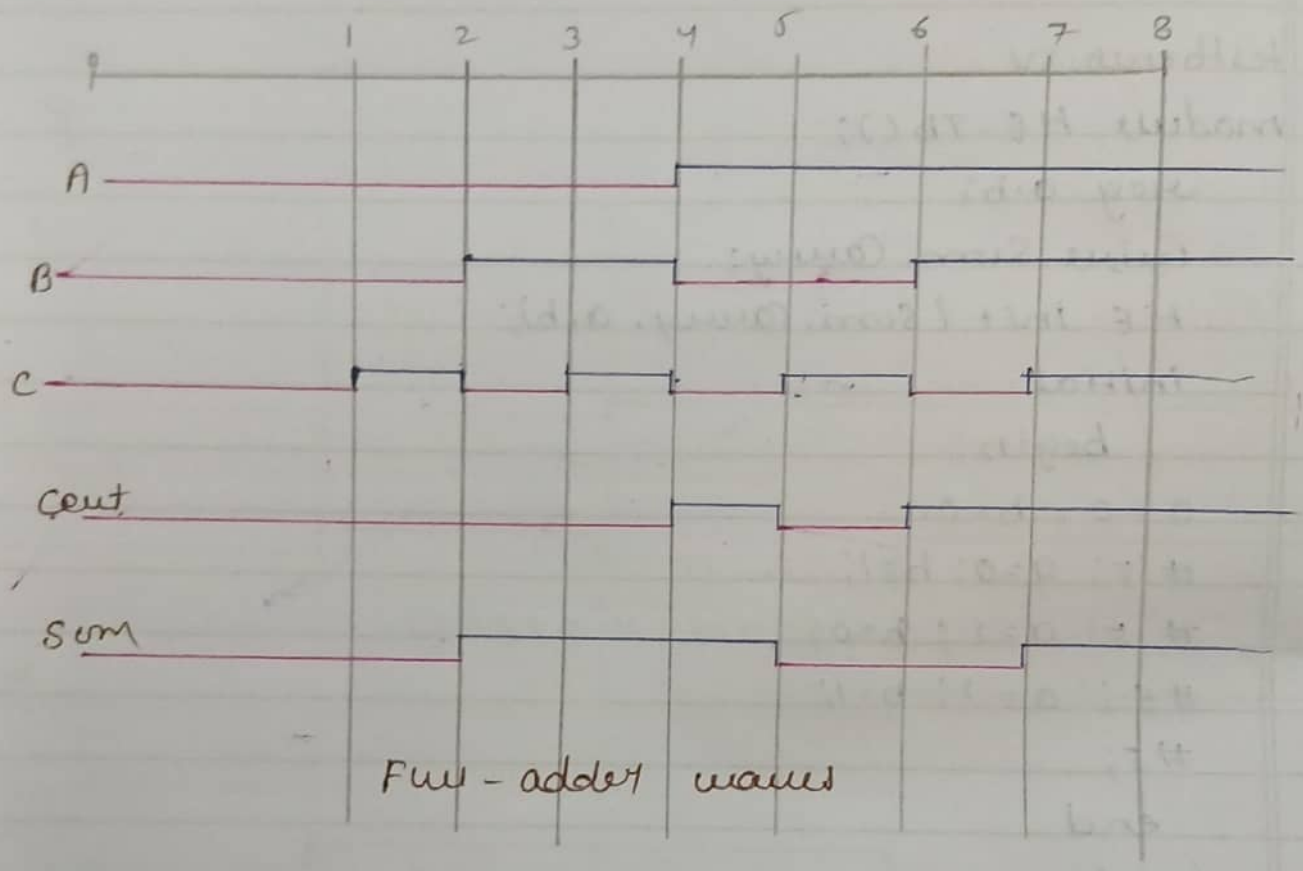
testbench.sv

```
module HE_Tb();
    reg a,b;
    wire Sum, Carry;
    HE inst (Sum, Carry, a, b);
    initial
        begin
    a=0; b=0;
    #5; a=0; b=1;
    #5; a=1; b=0;
    #5; a=1; b=1;
    #5;
     end
initial
    begin
     $dumpfile("dump.vcd");
     $dumpvars(1);
    end
end module
```

design.sv.

```
module HE (Sum, Carry, a, b);
    output sum, Carry;
    input a, b;
    assign sum = a^b;
    assign Carry = a&b;
end module
```


Scanned with CamScanner



Full - adder waves

Scanned with CamScanner

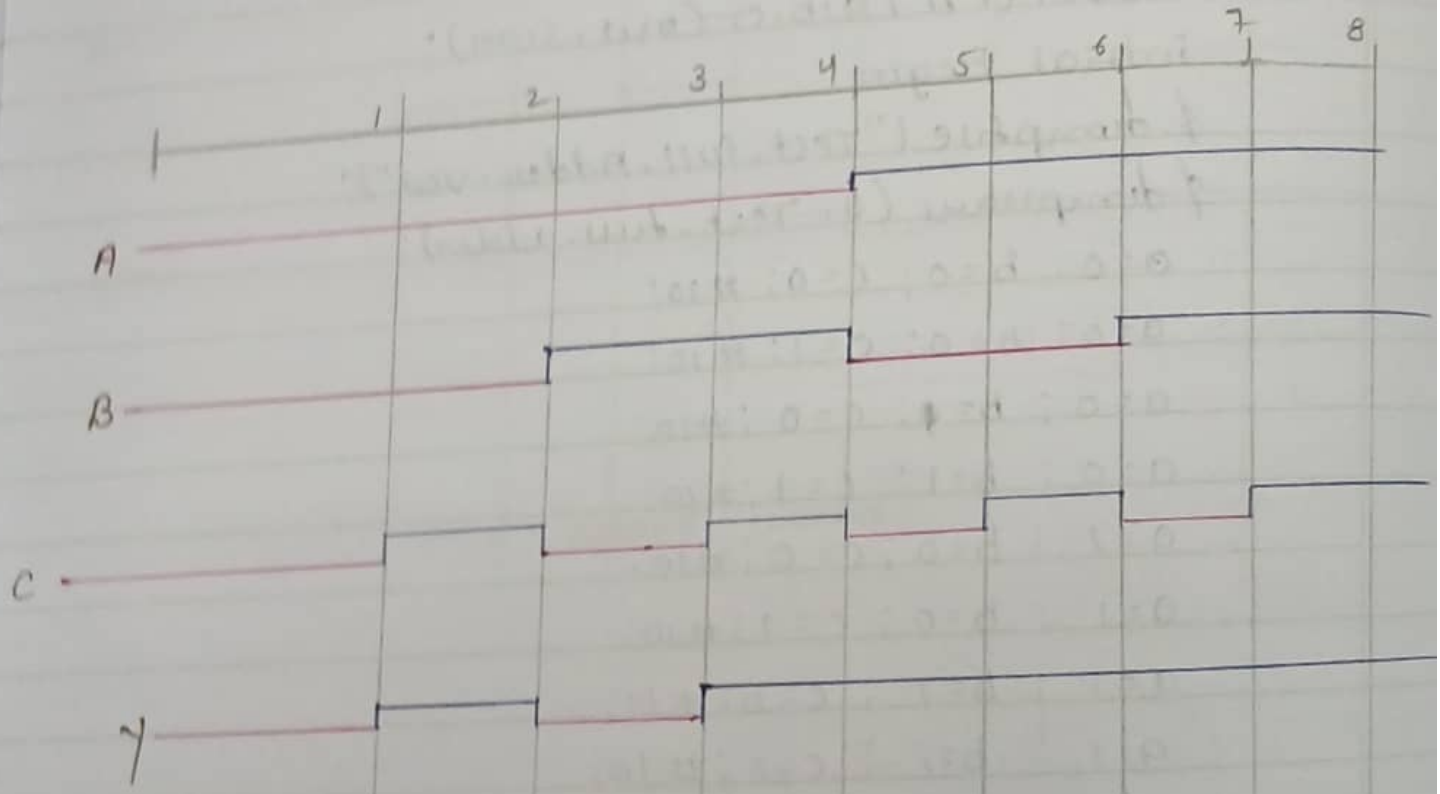
* Full Adder

```
module Test_full_Adder;
    reg a = 0;
    reg b = 0;
    reg c = 0;
    wire sum, Cout;
    Full_Adder (FA / a, b, c, Cout, sum);
        initial begin
        $ dumpfile ("Test_full_Adder.vcd");
        $ dumpvars (0, Test_full_adder);
         a=0; b=0; c=0; #10;
         a=0; b=0; c=1; #10;
         a=0; b=1; c=0; #10;
         a=0; b=1; c=1; #10;
         a=1; b=0; c=0; #10;
         a=1; b=0; c=1; #10;
         a=1; b=1; c=0; #10;
         a=1  b=1; c=1; #10;
         $ finish
        end
end module
```

design.sv

```
module full_adder (input a, b, c, output Cout, sum);
    assign # 6 sum = (a^b)^c;
    assign # 7 Cout = a&b | (a^b) & c;
endmodule
```


Scanned with CamScanner



EP waves of function

$F(A,B,C) = A + BC + \bar{A}C$

Scanned with CamScanner

POORNIMA

EXPERIMENT - 3

* OBJECTIVE: Design $F(A,B,C) = A + BC + \bar{A}C$

testbench.sv

```
module testbench;
  logic A,B,C;
  logic F;
  design dut(.A(A), .B(B), .C(C), .F(F));

  initial begin
    $dumpfile("wave.vcd")
    $dumpvars(0, testbench);

      A=0; B=0; C=0; #10;
      A=0; B=0; C=1; #10;
      A=0; B=1; C=0; #10;
      A=0; B=1; C=1; #10;
      A=1; B=0; C=0; #10;
      A=1; B=0; C=1; #10;
      A=1; B=1; C=0; #10;
      A=1; B=1; C=1; #10;
      $finish;
      end
endmodule
```

Design.sv

```
module design(
    input logic A,
    input logic B,
    input logic C,
```


Scanned with CamScanner

POORNIMA

```
    output logic F
);
  always_comb begin
        F = A|(B&C)|(~A&C);
  end
end module.
```


Scanned with CamScanner